# Layer10 Project Report

*Grounded Long-Term Memory via Structured Extraction, Union-Find Deduplication, and a Context Knowledge Graph*

**Divyaansh Narkhede**
**Roll No. : 24B091**

System Architecture & Implementation Report

March 2026

**Abstract**

This report presents a comprehensive technical examination of the **Layer10 Grounded Long-Term Memory System** — a fully local, privacy-preserving pipeline that converts an unstructured email corpus into a rigorously evidenced knowledge graph. The system combines a local Large Language Model (`llama3.2:3b` via Ollama) with deterministic rule-based fallback extractors to produce typed entities and evidenced relational claims. These raw extractions are progressively refined through a three-layer deduplication framework: content-hash artifact elimination, Union-Find entity canonicalization backed by `all-MiniLM-L6-v2` semantic embeddings, and temporal claim super-session. The final graph is serialized as a NetworkX `MultiDiGraph` and exposed via an interactive Streamlit application featuring an embedded `vis-network.js` physics visualization, semantic retrieval, and a full merge-audit trail. Every design decision prioritizes *grounded retrieval*: no claim exists in the graph without a verbatim evidence pointer storing exact character offsets back to the originating email byte sequence.

## Contents

**1 Introduction and Motivation** **3**
1.1 The Enron Email Dataset . . . 3
1.2 System Goals and Design Principles . . . 4

**2 Ontology and Schema Design (`schema.py`)** **4**
2.1 Entity Model . . . 4
2.2 Evidence and Grounding . . . 5
2.3 Claim Model . . . 5
2.4 MergeRecord and Reversibility . . . 6
2.5 MemoryStore Container . . . 7

**3 Configuration Layer (`config.py`)** **7**

**4 Corpus Ingestion** (`01_download_corpus.py`) **7**

**5 Structured Extraction** (`02_extract.py`) **8**
5.1 System Prompt Engineering . . . . . 8
5.2 Pydantic Validation Loop . . . . . 9
5.3 Deterministic Rule-Based Fallback . . . . . 9

**6 Deduplication and Canonicalization** (`03_dedup.py`) **9**
6.1 Layer 1 — Artifact Deduplication . . . . . 9
6.2 Layer 2 — Entity Canonicalization via Union-Find . . . . . 10
6.3 Layer 3 — Claim Deduplication and Temporal Super-session . . . . . 10

**7 Memory Graph Construction** (`04_graph.py`) **11**

**8 Semantic Retrieval** (`05_retrieve.py`) **12**

**9 Interactive Application** (`06_app.py`) **12**

**10 Exhaustive Setup and Execution Guide** **13**
10.1 Prerequisites . . . . . 13
10.2 Step 1 — Install Ollama . . . . . 13
10.3 Step 2 — Clone Repository and Set Up Python Environment . . . . . 14
10.4 Step 3 — (Optional) Place the Enron Archive . . . . . 14
10.5 Step 4 — Run the Full Pipeline . . . . . 14
10.6 Step 5 — Launch the Interactive UI . . . . . 15
10.7 Step 6 — Run a Semantic Query (CLI) . . . . . 15
10.8 Troubleshooting . . . . . 15

**11 Output Artifacts** **15**

**12 Conclusion** **16**

# 1 Introduction and Motivation

Modern Large Language Models lack persistent, verifiable memory. When facts change — a person's role, a project's status, a financial instrument's value — the model has no mechanism to record the transition, cite the source, or resolve the conflict. The Layer10 system is built to solve this problem for a realistic, deeply noisy corpus: the Enron email archive.

The core thesis is that a *grounded* memory graph — one where every node and every edge is anchored to an exact textual excerpt at a precise byte offset — is fundamentally more trustworthy than either a raw vector store (which loses structure) or a generic knowledge graph (which loses provenance). Layer10 enforces this contract at every stage of the pipeline.

## 1.1 The Enron Email Dataset

The **Enron Email Corpus** is a canonical NLP and network-analysis benchmark. It was made public by the U.S. Federal Energy Regulatory Commission (FERC) during its investigation into Enron Corporation's 2001 bankruptcy, one of the largest corporate collapses in history. Key statistics:

- $\approx$ **500,000 messages** authored by $\approx$ **150 senior employees**.
- Date range spanning **1999–2002**, covering the full arc from growth to collapse.
- Messages distributed across **158 named mailbox directories** in the CMU Maildir format.
- The corpus has been used in fraud detection, social network analysis, authorship attribution, and information extraction research.

For Layer10, the Enron dataset provides three categories of challenge that stress-test every component:

1. **Dynamic Organizational Hierarchy**: Promotions (e.g. Jeff Skilling from President to CEO), role changes, and reporting reassignments create *temporal conflicts* that test the system's claim super-session logic. A fact that was `CURRENT` in January 2001 may be `HISTORICAL` by April.
2. **Non-Standard Domain Entities**: The corpus is dense with proprietary terminology — special purpose vehicles (*Raptor I/II/III/IV*, *Chewco*, *LJM Cayman*), internal systems (*EnronOnline*, *RiskTrak*), and projects (*Broadband*) — that do not appear in pre-trained model vocabularies and require custom ontologies.
3. **Extreme Conversational Noise**: Deeply nested forward/reply chains (> > >), mixed encodings, legal disclaimers repeated in every email, and varying date formats (RFC 2822 mixed with informal styles) challenge the artifact deduplication and evidence grounding layers specifically.

**Dataset Sourcing**

The system reads the CMU Maildir directly from `data/maildir/` (subdirectory per user). Thread reconstruction uses the RFC 2822 `In-Reply-To`, `References`, and `Message-ID` headers. When the full archive is unavailable, a hardcoded synthetic corpus of **75 emails** across structured threads is used as a deterministic fallback, producing reproducible test

graphs.

### 1.2 System Goals and Design Principles

**G1. Grounded provenance**: every claim stores a verbatim quote $\leq 300$ characters and exact character offsets (`char_offset_start, char_offset_end`).

**G2. Fully local execution**: Ollama serves the LLM at `localhost:11434`; no data leaves the machine.

**G3. Reversibility**: all merges emit a `MergeRecord` containing pre-merge snapshots, enabling lossless undo.

**G4. Incremental resumability**: each pipeline stage checkpoints its output to `output/`; a crash can resume from the last saved state.

**G5. Strict typing**: all inter-module data is validated via Pydantic v2 models; no untyped dicts cross module boundaries.

## 2 Ontology and Schema Design (`schema.py`)

All data structures are defined in `schema.py` as Pydantic v2 `BaseModel` subclasses. This single-source-of-truth design guarantees that the LLM, the deduplicator, the graph builder, and the UI all operate on identical validated objects.

### 2.1 Entity Model

An `Entity` represents a canonical node in the knowledge graph. The ontology is intentionally constrained to **eight types** (via `EntityType` enum) to prevent schema drift while covering all meaningful Enron domain concepts:

| Type | Example | Purpose |
|---|---|---|
| `Person` | *Kenneth Lay, Jeff Skilling* | Personnel, executives |
| `Organisation` | *Enron, Arthur Andersen, FERC* | Firms, regulators, counterparties |
| `FinancialInstrument` | *Raptor, Chewco, LJM Cayman* | SPEs, funds, instruments |
| `Event` | *Bankruptcy filing, Board Meeting* | Bounded occurrences |
| `Project` | *EnronOnline, Broadband* | Internal initiatives |
| `System` | *RiskTrak, SAP* | Software / IT platforms |
| `Role` | *CEO, CFO, Treasurer* | Organisational positions |
| `Location` | *Houston, Portland* | Geographic entities |

Table 1: Entity type ontology with Enron-domain examples

Key fields on the `Entity` model:

```
class Entity(BaseModel):
    id: str = Field(default_factory=lambda: uuid.uuid4().hex[:12])
    name: str
    type: EntityType
```

```
    aliases: list[str] = Field(default_factory=list)
    first_seen: Optional[str] = None
    last_seen:  Optional[str] = None
    metadata:   dict = Field(default_factory=dict)

    def canonical_key(self) -> str:
        return f"{self.type.value}::{self.name.lower().strip()}"
```

Listing 1: Entity model excerpt (schema.py)

The `canonical_key()` method forms the basis for Layer 2 exact-match deduplication: two entities sharing the same lowercased name and type will collapse into one regardless of their `id`.

## 2.2 Evidence and Grounding

Every claim is anchored by one or more `Evidence` objects:

```
class Evidence(BaseModel):
    source_id:          str             # email_id (unique per message)
    excerpt:            str             # verbatim quote, <= 300 chars
    timestamp:          Optional[str]   # ISO-8601 event time
    source_type:        str = "email"
    author:             Optional[str]
    subject:            Optional[str]
    char_offset_start:  Optional[int]   # byte offset in source
    char_offset_end:    Optional[int]   # byte offset in source

    def fingerprint(self) -> str:
        raw = f"{self.source_id}|{self.excerpt[:100]}"
        return hashlib.md5(raw.encode()).hexdigest()
```

Listing 2: Evidence model (schema.py)

The pair `(char_offset_start, char_offset_end)` makes every evidence pointer a deterministic *bookmark* into the raw email body. The `fingerprint()` method is used to deduplicate identical evidence objects that appear when the same email is processed in multiple threads.

## 2.3 Claim Model

A `Claim` is a typed, evidenced, time-bounded assertion linking two entities. Unlike a simple RDF triple, it carries rich temporal and provenance metadata:

```
class Claim(BaseModel):
    id:            str = Field(default_factory=lambda: uuid.uuid4().hex
   [:12])
    subject_id:    str;   subject_name: str
    relation:      RelationType
    object_id:     str;   object_name:  str
    confidence:    float = Field(ge=0.0, le=1.0, default=0.8)
    status:        ClaimStatus = ClaimStatus.CURRENT
    evidence:      list[Evidence] = Field(default_factory=list)
    valid_from:    Optional[str] = None
    valid_until:   Optional[str] = None   # set when superseded
    superseded_by:Optional[str] = None   # claim id of replacement

    def canonical_key(self) -> str:
```

```
        return (f"{self.subject_name.lower().strip()}|"
                f"{self.relation.value}|"
                f"{self.object_name.lower().strip()}")
```

Listing 3: Claim model (schema.py)

**Relation types** are organized into four semantic groups across the `RelationType` enum (25 values):

- *Organizational*: `works_at`, `has_role`, `reports_to`, `manages`, `member_of`
- *Actions*: `created`, `sent_email`, `decided`, `proposed`, `approved`, `rejected`, `warned_about`, `investigated`
- *States/Facts*: `has_status`, `has_value`, `depends_on`, `conflicts_with`, `supersedes`, `related_to`
- *Financial*: `invested_in`, `profited_from`, `owes`, `acquired`

`ClaimStatus` governs the temporal lifecycle:

| Status | Semantics |
|---|---|
| `CURRENT` | The assertion is the most recently confirmed truth. |
| `HISTORICAL` | Superseded by a newer claim; `superseded_by` points to the replacement. |
| `RETRACTED` | The source email was deleted, redacted, or explicitly withdrawn. |
| `DISPUTED` | Conflicting evidence exists; the claim's confidence is penalized. |

Table 2: ClaimStatus lifecycle

## 2.4 MergeRecord and Reversibility

Every structural mutation to the graph emits a `MergeRecord`:

```
class MergeRecord(BaseModel):
    id:           str
    merge_type:   str       # "entity" or "claim"
    winner_id:    str
    loser_id:     str
    reason:       str       # e.g. "exact_match" or "semantic_similarity
   =0.93"
    timestamp:    str
    reversible:   bool = True
    original_snapshots: dict  # pre-merge JSON snapshots of winner +
   loser
```

Listing 4: MergeRecord model (schema.py)

Because `original_snapshots` stores the full pre-merge state of both participants, any merge can be replayed in reverse: restore the loser entity, update all `subject_id`/`object_id` references, and delete the merge record. The UI exposes this trail in the "Merges" audit tab.

### 2.5 MemoryStore Container

`MemoryStore` is the first-class container exposing indexed access, helpers, and serialization for the entire graph state:

- Entities and Claims are stored as `dict[str, Entity/Claim]` keyed by ID — $O(1)$ lookups.
- `find_entities_by_name()` performs case-insensitive substring search over both canonical name and alias pool.
- `get_claims_for_entity()` and `get_claims_between()` support focused graph traversal without loading the full NetworkX structure.
- `remove_orphan_entities()` prunes nodes unreferenced by any claim, keeping the graph clean after merges.
- `to_snapshot()` / `from_snapshot()` serialize to/from the flat `MemoryGraphSnapshot` format for JSON persistence.

## 3 Configuration Layer (`config.py`)

All tunable parameters and path constants live in `config.py`, imported by every stage. Critical values:

| Constant | Value | Effect |
|---|---|---|
| OLLAMA_MODEL | llama3.2:3b | Generative model for extraction |
| OLLAMA_HOST | http://localhost:11434 | Local REST endpoint |
| EMBEDDING_MODEL | all-MiniLM-L6-v2 | Entity similarity embeddings |
| SIMILARITY_THRESHOLD | 0.85 | Cosine threshold for entity merge |
| MAX_RETRIES | 3 | JSON repair attempts per thread |
| CONFIDENCE_THRESHOLD | 0.5 | Minimum confidence to retain a claim |
| BATCH_SIZE | 5 | Emails per inference batch |
| MIN_MESSAGES_PER_THREAD | 5 | Thread floor (context richness) |
| MAX_MESSAGES_PER_THREAD | 15 | Thread ceiling (context window guard) |
| TARGET_THREAD_COUNT | 15 | Threads submitted to extraction |

Table 3: Key configuration constants and their pipeline roles

The module also calls `os.makedirs(DATA_DIR, exist_ok=True)` and `os.makedirs(OUTPUT_DIR, exist_ok=True)`, guaranteeing that the `data/` and `output/` directories are created on first import — eliminating setup friction.

## 4 Corpus Ingestion (`01_download_corpus.py`)

The ingestion stage transforms raw email files into a list of **conversation threads**, each bounded by `[MIN, MAX]_MESSAGES_PER_THREAD`. The output is `data/enron_raw_emails.json`.

1. **Maildir Walk**: Recursively walks `data/maildir/` collecting all `.txt` files.

2. **RFC 2822 Parse**: Each file is parsed for the standard headers: `Message-ID`, `In-Reply-To`, `References`, `From`, `To`, `Date`, `Subject`, plus the raw body after the double-CRLF boundary.
3. **Thread Assembly**: A parent-map is built from `In-Reply-To` pointers. Emails are clustered into trees; each tree becomes a thread. Threads outside the `[MIN, MAX]` bounds are discarded.
4. **Body Sanitization**: Nested reply indicators (> prefix lines), legal footers, and MIME headers are stripped before storage, reducing noise in the LLM context window.
5. **Synthetic Fallback**: If fewer than `TARGET_THREAD_COUNT` real threads are found, the script appends synthetic threads — 75 hand-authored emails covering key Enron events — to reach the target.

# 5 Structured Extraction (`02_extract.py`)

`02_extract.py` is the most computationally intensive stage. For each thread it dispatches an LLM call and validates the response, with deterministic fallback.

## 5.1 System Prompt Engineering

The LLM is given a two-part payload: a `SYSTEM_PROMPT` (constant, MD5-hashed as `PROMPT_HASH` for versioning) and a `THREAD_PROMPT` assembled per thread. The system prompt instructs the model to:

- Return **only** a valid JSON object — no markdown fences, no commentary.
- Use **only** the 8 entity types and 25 relation types defined in the ontology.
- Anchor every claim with an `email_id` and a verbatim `quote` ($\leq 200$ chars).
- Calibrate confidence: `1.0` = explicitly stated, `0.7` = strongly implied, `0.5` = loosely implied.
- Track *temporal evolution* across the thread — if a fact changes between emails, emit separate claims.

```
THREAD_PROMPT_TEMPLATE = """Extract entities and claims from this email
    thread:

Thread Subject: {subject}
Thread ID: {thread_id}
Message Count: {message_count}
Participants: {participants}

--- EMAILS IN CHRONOLOGICAL ORDER ---
{emails_text}
--- END OF THREAD ---
"""
```

Listing 5: Thread prompt template (02_extract.py)

Each email in the thread is formatted with its `email_id`, sender, recipient, timestamp, subject, and body (truncated to 1000 characters to respect the context window).

## 5.2 Pydantic Validation Loop

After receiving the raw LLM string, the parser applies a three-attempt validation loop with progressive JSON repair:

**Attempt 1:** Direct `json.loads()` of the raw response.

**Attempt 2:** Regex extraction of the outermost `{...}` block (handles preamble/postamble hallucinations).

**Attempt 3:** Truncation to the last valid `}` character (handles truncated responses from slow inference).

After successful parse, each entity and claim is constructed as a validated Pydantic object. Evidence `char_offset_start/end` are computed by searching the original email body for the LLM-returned `quote` using Python's `str.find()`, providing exact byte pointers. Claims below `CONFIDENCE_THRESHOLD` (0.5) are silently dropped.

The result is an `ExtractionResult` object stored per-thread, all serialized to `output/extractions.json`.

## 5.3 Deterministic Rule-Based Fallback

When the LLM is offline (`DISABLE_OLLAMA` env var) or all retries fail, a **deterministic regex extractor** runs on the raw email bodies. It targets:

- **Monetary values**: `$[0-9,]+(.[0-9]+)?[MBK]?` — creates a `has_value` claim with the matched amount as object.
- **Known entity names**: a hardcoded dictionary of Enron executives, SPEs, and regulators.
- **Organizational patterns**: regex for *"X reports to Y"*, *"X is the [role] of [org]"*.

This guarantees that every thread produces *some* structured output regardless of model availability, making the full pipeline runnable without a GPU.

# 6 Deduplication and Canonicalization (`03_dedup.py`)

Naive extraction over 15 overlapping threads produces hundreds of duplicate or near-duplicate entities and claims. `03_dedup.py` collapses these into a canonical graph through three sequentially applied layers.

## 6.1 Layer 1 — Artifact Deduplication

The first layer operates at the extraction-result (per-email) level. Because Enron threads contain heavily quoted previous messages, many emails share near-identical bodies. The algorithm:

1. Extracts the first 150 normalized characters of all evidence excerpts from an `ExtractionResult`.
2. Lowercases and strip-normalizes the content to form a `norm_key`.
3. If the key has been seen before, the *current* extraction's entities and claims are **merged into the existing record** rather than discarded — preserving any unique extractions while avoiding LLM re-calls.
4. Reports the count of merged duplicates to stdout for monitoring.

## 6.2 Layer 2 — Entity Canonicalization via Union-Find

Entity canonicalization resolves surface-form variants of the same real-world entity (e.g. "K. Lay", "Kenneth L. Lay", "Ken Lay" → *Kenneth Lay*).

**Data structure.** The `UnionFind` class implements a disjoint-set forest with path compression:

```
class UnionFind:
    def __init__(self):
        self.parent = {}

    def find(self, x):
        if x not in self.parent:
            self.parent[x] = x
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]  # path compression
            x = self.parent[x]
        return x

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra != rb:
            self.parent[rb] = ra   # a becomes canonical root
            return True
        return False
```

Listing 6: UnionFind with path compression (03_dedup.py)

The amortized time complexity for $m$ operations on $n$ entities is $O(m \cdot \alpha(n))$, where $\alpha$ is the inverse Ackermann function — effectively $O(1)$ per operation at any realistic scale.

**Two-pass merge strategy.** Within each `EntityType` group:

1. **Exact match**: Entities sharing the same `canonical_key()` (lowercased name + type) are immediately unioned.
2. **Semantic similarity**: For all unmerged pairs, names (plus alias lists) are encoded using `SentenceTransformer("all-MiniLM-L6-v2")` into 384-dimensional vectors. Pairs with cosine similarity $\geq 0.85$ are unioned.

$$\text{cosine_sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\| + \varepsilon}$$

After all unions are established, canonical representatives are selected (lowest UUID, or the entity with the most evidence). All `subject_id` and `object_id` references in claims are rewritten to point to the canonical ID. Aliases are accumulated in the winner's `aliases` list. Every merge emits a `MergeRecord` with `reason="exact_match"` or `reason=f"semantic_similarity={score:.2f}"`.

## 6.3 Layer 3 — Claim Deduplication and Temporal Super-session

After entity canonicalization, many claims become identical when their `canonical_key()` is re-evaluated (since `subject_name` and `object_name` now point to canonical forms). The algorithm:

1. Groups claims by their `canonical_key()` = `subject|relation|object`.

2. Within each group, identifies the claim with the most recent `valid_from` as the *winner*.
3. Evidence lists from all duplicates are merged into the winner (deduped by `Evidence.fingerprint()`).
4. Older duplicate claims are set to `status = HISTORICAL` with `superseded_by = winner.id`.
5. A `MergeRecord` of type `"claim"` is emitted for each collapse.

This mechanism naturally handles role transitions: when Jeff Skilling moves from `has_role: President` to `has_role: CEO`, both claims share the same subject/object pattern but different relations — they are kept as separate claims. But if two threads both extract "Jeff Skilling has_role President" with different timestamps, only the most recent survives as `CURRENT`; the older becomes `HISTORICAL`.

# 7 Memory Graph Construction (`04_graph.py`)

`04_graph.py` translates the canonicalized `MemoryStore` into a NetworkX `MultiDiGraph` and computes summary statistics.

**Graph structure:**

- **Nodes** = canonical `Entity` objects. Node attributes: `name`, `type`, `aliases`, `first_seen`, `last_seen`, `metadata`.
- **Edges** = `Claim` objects. A `MultiDiGraph` is used (not a simple `DiGraph`) because two entities can have multiple relations of different types simultaneously. Edge key = `claim.id`. Edge attributes include the full serialized evidence list.

```
G = nx.MultiDiGraph()
for ent in entities:
    G.add_node(ent.id, name=ent.name, type=ent.type.value,
               aliases=ent.aliases, ...)
for claim in claims:
    G.add_edge(claim.subject_id, claim.object_id,
               key=claim.id,
               relation=claim.relation.value,
               confidence=claim.confidence,
               status=claim.status.value,
               evidence=[e.model_dump() for e in claim.evidence],
               ...)
```

Listing 7: Graph construction core (04_graph.py)

**Graph statistics** computed by `graph_stats()` and saved to `output/graph_stats.json`:

- Node / edge counts, node type distribution, relation type distribution.
- Claim status distribution (`current` vs. `historical`).
- **Top 15 entities by degree** — the most connected individuals/organisations.
- **Evidence coverage**: percentage of claims that have at least one evidence pointer.
- Number of weakly connected components (reveals isolated sub-graphs from sparse threads).

Two serializations are written:

- `output/memory_graph.json`: the full `MemoryGraphSnapshot` (entities + claims + merge log).
- `output/memory_graph_nx.json`: the NetworkX graph serialized via `nx.node_link_data()` for optional downstream graph-algorithm use.

# 8 Semantic Retrieval (`05_retrieve.py`)

The retrieval module converts a natural-language question into a grounded **Context Pack** — a ranked list of claims with evidence snippets that is passed to the LLM for answer generation.

**Retrieval pipeline (5 stages):**

**Stage 1:** **Query Embedding**: The question is encoded by `all-MiniLM-L6-v2` into a 384-d vector $\mathbf{q}$.

**Stage 2:** **Entity Scoring**: Each entity's name and alias list is concatenated and encoded. Cosine similarity between $\mathbf{q}$ and each entity vector selects the `top_k` (default: 10) seed entities.

**Stage 3:** **1-Hop Graph Expansion**: For each seed entity, all incident edges (both in and out) are collected. Then all neighbors of seed entities are added, and their edges are collected. This *1-hop expansion* brings in context that the question didn't explicitly name.

**Stage 4:** **Claim Ranking**: Each candidate claim is encoded (`subject relation object` as text). A composite score is computed:

$$\text{score} = 0.6 \cdot \text{sim}(\mathbf{q}, \mathbf{c}) + 0.2 \cdot \text{confidence} + 0.2 \cdot \text{status_boost}$$

where `status_boost` = 1.0 for `CURRENT` claims and 0.7 for `HISTORICAL/DISPUTED`.

**Stage 5:** **Evidence Assembly**: Evidence snippets are de-duplicated by fingerprint and sorted by claim score. Each snippet includes `source_id`, `excerpt`, `timestamp`, `author`, `subject`, and the human-readable `supporting_claim` string.

The output **Context Pack** is a structured dict with `matched_entities`, `relevant_claims`, `evidence_snippets`, and a `context_summary` string. This pack is saved to `output/context_packs.json` and also passed directly to Ollama to generate a cited answer.

# 9 Interactive Application (`06_app.py`)

The Streamlit application provides five navigation pages via `st.sidebar.radio`:

1. **Graph**: Full interactive knowledge graph rendered via `vis-network.js` 9.1.2 (bundled locally in `lib/vis-9.1.2/`).
2. **Entities**: Searchable, filterable table of all canonical entities.
3. **Claims**: Filterable claims table with confidence sliders and status filter.
4. **Query**: Natural-language QA with grounded citations.
5. **Merges**: Audit trail of all `MergeRecords` with expand-to-snapshot buttons.

**Graph physics configuration.** Disconnected nodes in large graphs tend to drift indefinitely. The embedded JS options use the `barnesHut` solver with tuned parameters to prevent this:

```
physics: {
    barnesHut: {
        gravitationalConstant: -8000,
        centralGravity: 0.8,
        springLength: 120,
        damping: 0.3
    }
}
// Freeze layout after stabilization
network.on("stabilized", function() {
    network.setOptions({physics: false});
});
```

Listing 8: vis-network.js physics config (06_app.py)

The `stabilized` event listener disables physics after the initial layout converges — preventing indefinite drift while still allowing manual drag-and-drop repositioning.

# 10 Exhaustive Setup and Execution Guide

## 10.1 Prerequisites

- **OS**: Linux (tested), macOS, or Windows with WSL2.
- **Python**: 3.10 or higher (3.11 recommended).
- **RAM**: 8 GB minimum; 16 GB recommended for embedding large graphs.
- **Disk**: 3 GB for `llama3.2:3b` model weights + ~1 GB for the Enron archive.
- **GPU** (optional but recommended): Any CUDA 11.8+ or Apple Silicon GPU significantly accelerates extraction.

## 10.2 Step 1 — Install Ollama

```
# Official one-line installer (Linux / macOS)
curl -fsSL https://ollama.com/install.sh | sh

# Start the local LLM server (keep this terminal open, or run as a
   service)
ollama serve

# Pull the extraction model (approx 2 GB download)
ollama pull llama3.2:3b

# Verify the model is available
ollama list
```

*Tip*: To run Ollama as a persistent systemd service on Linux:

```
sudo systemctl enable ollama
sudo systemctl start ollama
```

## 10.3 Step 2 — Clone Repository and Set Up Python Environment

```
git clone <repository-url>
cd layer10_new

# Create an isolated virtual environment
python3 -m venv .venv
source .venv/bin/activate   # Windows WSL: same command

# Upgrade pip and install all dependencies
pip install --upgrade pip
pip install -r requirements.txt
```

The `requirements.txt` pins:

| Package | Minimum Version |
|---|---|
| `ollama` | 0.4.0 |
| `pydantic` | 2.0 |
| `sentence-transformers` | 2.2.0 |
| `networkx` | 3.0 |
| `pyvis` | 0.3.2 |
| `streamlit` | 1.30.0 |
| `numpy` | 1.24.0 |

Table 4: Python dependencies

## 10.4 Step 3 — (Optional) Place the Enron Archive

```
# Download the CMU Maildir archive (~423 MB)
# From: https://www.cs.cmu.edu/~enron/enron_mail_20150507.tar.gz
wget https://www.cs.cmu.edu/~enron/enron_mail_20150507.tar.gz -P data/
tar -xzf data/enron_mail_20150507.tar.gz -C data/
# Result: data/maildir/<username>/ directories
```

*If this step is skipped*, the pipeline automatically falls back to the 75-email synthetic corpus.

## 10.5 Step 4 — Run the Full Pipeline

```
# Run all 4 stages sequentially (recommended for first run)
python run_pipeline.py
```

Or execute each stage independently to inspect intermediate outputs:

```
# Stage 1: Ingest corpus -> data/enron_raw_emails.json
python 01_download_corpus.py

# Stage 2: LLM extraction -> output/extractions.json
# NOTE: This is the slowest step. Without GPU, allow 10-30 min.
python 02_extract.py

# Stage 3: Deduplicate and canonicalize -> output/deduped.json
python 03_dedup.py
```

```
# Stage 4: Build NetworkX graph -> output/memory_graph.json
python 04_graph.py
```

### 10.6 Step 5 — Launch the Interactive UI

```
streamlit run 06_app.py
```

Open http://localhost:8501 in a Chromium-based browser. Firefox is supported but Chromium provides superior Canvas rendering performance for the `vis-network.js` graph.

### 10.7 Step 6 — Run a Semantic Query (CLI)

```
# Direct CLI query (no UI required)
python 05_retrieve.py "Who did Jeff Skilling report to?"
```

The script prints the context pack and, if Ollama is running, a grounded LLM answer with inline citations.

### 10.8 Troubleshooting

| Symptom | Cause | Fix |
|---|---|---|
| `ConnectionRefusedError` on port 11434 | Ollama not running | `ollama serve` |
| `model not found` | Model not pulled | `ollama pull llama3.2:3b` |
| Extraction produces empty graph | All retries failed, fallback ran | Check `output/extractions.j` |
| `CUDA out of memory` | Model too large for VRAM | Use `ollama pull llama3.2:1` |
| Graph node drift in UI | Physics not stabilized | Reload page; check JS console |

Table 5: Common issues and resolutions

## 11 Output Artifacts

After a successful pipeline run, the `output/` directory contains:

| File | Contents |
|---|---|
| `extractions.json` | Per-thread `ExtractionResult` list (raw, pre-dedup). |
| `deduped.json` | Canonicalized `MemoryGraphSnapshot` with merge log. |
| `memory_graph.json` | Final graph snapshot (entities + claims + merges). |
| `memory_graph_nx.json` | NetworkX node-link JSON for external graph tools. |
| `graph_stats.json` | Summary statistics (node counts, top entities, coverage). |
| `context_packs.json` | Saved context packs from previous retrieval queries. |

Table 6: Pipeline output artifacts

# 12 Conclusion

The Layer10 system demonstrates that a rigorous *grounded* approach to long-term memory — enforcing verbatim evidence anchoring, typed schemas, reversible merges, and temporal claim super-session — produces knowledge graphs that are fundamentally more trustworthy than retrieval systems that operate over raw vector embeddings alone.

Key technical contributions:

- A **fully local** extraction pipeline using Ollama, providing privacy guarantees absent from cloud API approaches.
- **Character-offset evidence pointers** that enable byte-exact source attribution for every claim.
- A **Union-Find canonicalization** layer backed by sentence-embedding similarity, resolving surface-form variants at sub-linear amortized cost.
- A **temporal claim lifecycle** model (`CURRENT` $\rightarrow$ `HISTORICAL` $\rightarrow$ `RETRACTED`) that cleanly captures organizational change over time.
- **Deterministic fallback extraction** ensuring pipeline completeness without GPU access.
- A **composite retrieval scoring function** that weights semantic similarity, claim confidence, and temporal recency for grounded QA.

Future directions include: multi-corpus ingestion (Slack, Linear, GitHub Issues) via pluggable corpus adapters; FAISS approximate nearest-neighbor indexing for sub-millisecond retrieval over millions of entities; and a web-hook based live-ingestion mode that incrementally updates the graph as new messages arrive.